

Flink 原理详解

第 2 讲：从 Source 到 Sink 的 DataStream 执行模型

Source → Transformation → keyBy → Operator → Sink
Parallelism → Subtask → Task → TaskManager → Task Slot

Flink DataStream：不是“读完再算”，而是事件不断流过算子



→ keyBy 后通常都需要网络重分区 →

本讲核心问题 你写下一个 Flink DataStream 程序后，它如何从一串 API 调用，变成多个机器上持续运行的并行任务？

面向零基础 | 强调底层原理、图解、误区、实验与练习

1. 这一讲学完，你应该真正理解什么

- DataStream 程序为什么可以看成一张持续运行的数据流图，而不是“一段从上到下执行完就退出的脚本”。
- Source、Transformation、keyBy、Window/State、Sink 分别在数据流中承担什么职责。
- Parallelism、Subtask、Task、Operator Chain、Task Slot、TaskManager 之间到底是什么关系。
- 为什么 keyBy 往往意味着数据需要跨网络重新分布，以及它和 MapReduce/Spark Shuffle 的联系。
- 一个 Flink Job 从客户端提交，到 JobManager 协调，再到 TaskManager 真正执行的完整生命周期。
- 为什么一个 Task Slot 不等于一个算子，也不等于一个线程；为什么默认 Slot Sharing 会让一个 Slot 承载一条流水线。

先建立一个总心智模型 Flink 不是“每来一条数据就临时启动一次程序”。Job 启动后，Source、算子和 Sink 对应的并行 Subtask 会长期存活，事件持续从上游流向下游。

2. DataStream 程序的第一张图：Source → Transformation → Sink

Flink DataStream：不是“读完再算”，而是事件不断流过算子

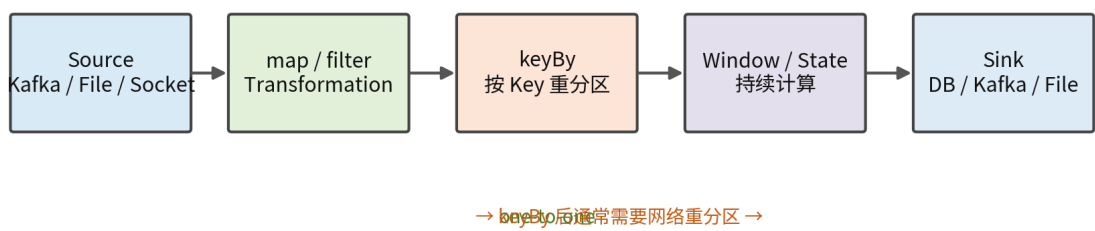


图 2-1 一个最基本的 Flink DataStream 流水线

在批处理里，我们习惯想“有一个文件，读完以后计算，再输出结果”。在流处理中，这种直觉需要改变：Source 可能永远持续产生事件，算子也会长期运行，Sink 则持续接收计算结果。

组件	通俗理解	典型例子
Source	数据从哪里进入 Flink	Kafka、文件、Socket、CDC、消息队列
Transformation	数据经过什么计算	map、filter、flatMap、聚合、窗口
keyBy	哪些事件以后必须由同一个逻辑分组处理	按 userId、deviceId、orderId 分区
State / Window	系统如何记住过去并按时间范围计算	累计金额、5 分钟窗口、会话状态
Sink	结果最终写到哪里	Kafka、数据库、文件、搜索引擎

核心概念 DataStream 不是“数据真的全部装在一个对象里”。它更像一份分布式数据流的描述：告诉

Flink 数据从哪里来、经过哪些算子、最后去哪里。

3. Transformation 与 Operator：每个算子都在做什么

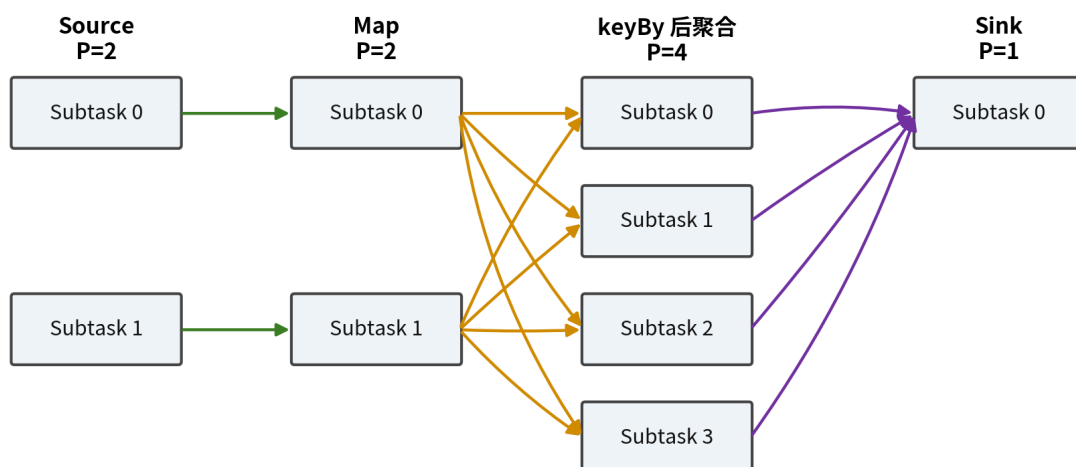
Transformation（转换）是你在程序里描述的数据处理步骤；运行时这些步骤会对应到 Operator（算子）及其并行实例。初学时可以把两者近似理解为“代码里的计算步骤”和“运行时真正执行这个步骤的组件”。

```
// 伪代码：只看数据流结构，不要求现在运行
source
  .filter(order -> order.isPaid())
  .map(order -> new Tuple2<>(order.userId, order.amount))
  .keyBy(x -> x.f0)
  .sum(1)
  .sinkTo(output);
```

- filter：一条记录进来，可能保留，也可能丢弃。
- map：一条记录进来，转换成另一条记录。
- keyBy：不负责求和；它负责把相同 Key 的事件路由到同一个下游逻辑分区。
- sum / reduce / process / window：真正对 Keyed Stream 进行聚合或状态计算。
- sink：把结果写出去；它通常也是整个数据流图的末端 Operator。

常见误区 keyBy ≠ groupBy 后把整组数据一次性交给你。流是持续到来的，所以 keyBy 更准确的理解是“持续路由”：每来一条事件，就决定它应该发给哪个下游 Subtask。

4. Parallelism 与 Subtask：Flink 是怎样并行起来的



Parallelism = 某个算子的并行 Subtask 数量；不同算子的并行度可以不同

图 4-1 不同 Operator 可以有不同并行度

官方定义中，一个 task/operator/source/sink 可以被拆成多个并行实例；这些并行实例就是 subtasks。一个算子的并行实例数量，就是它的 parallelism（并行度）。

术语	准确理解	不要误解成
Operator	一个计算步骤，例如 map 或 window	一台机器
Parallelism = 4	这个 Operator 有 4 个并行 Subtask	程序有 4 台服务器
Subtask	Operator 的一个并行执行实例	一个独立 Job
Stream Partition	上游并行实例输出的数据分片	HDFS Block

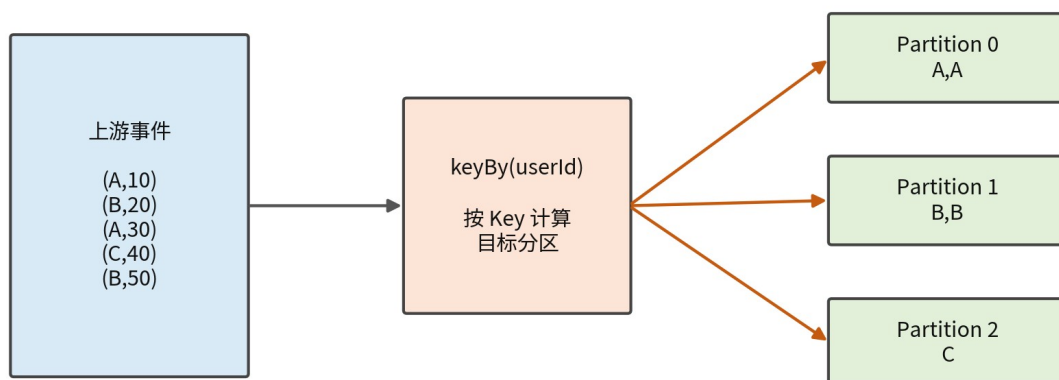
例子 如果 Source 并行度为 2，而聚合算子并行度为 4，那么 Source 只有 2 个 Subtask，但聚合阶段有 4 个 Subtask。Flink 不要求整个 Job 的每个算子并行度完全相同。

5. 数据在 Subtask 之间怎么流动：one-to-one 与 redistribution

Flink 的数据流边可以粗略分成两类：一类保持分区对应关系，一类会重新分配数据。理解这个区别，是理解 keyBy、网络开销和后续性能问题的基础。

模式	发生什么	典型情况
One-to-one / Forward	上游某个 Subtask 的数据直接流向对应下游 Subtask，顺序和分区关系基本保持	Source → map，且并行度兼容
Redistributing	记录需要根据规则重新路由到不同下游 Subtask，可能跨 TaskManager / 网络	keyBy、rebalance、rescale 等

keyBy 的核心保证：相同 Key 的事件进入同一个下游 Keyed Subtask



这与 MapReduce/Spark 的按 Key 重分区思想高度相似，但在流任务中持续发生。

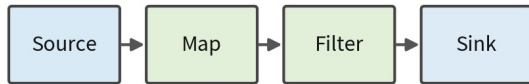
图 5-1 keyBy 把相同 Key 持续路由到同一个下游 Keyed Subtask

与前面课程的联系 MapReduce 的 Partition + Shuffle、Spark 的 partitionBy/reduceByKey、Flink 的 keyBy，在“按 Key 把相关数据送到一起”这一点上有共同思想；区别在于 Flink 的流式路由会随着事件持续

发生。

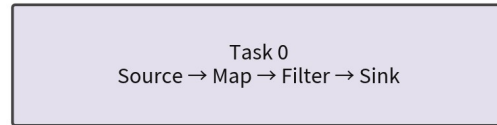
6. Subtask 不一定等于一个独立 Task：Operator Chaining

未链化：每个算子都可形成独立 Task



更多线程切换 / 缓冲

Operator Chaining：可链化的算子合成一个 Task / 线程



减少线程间交接和缓冲，通常吞吐更高、延迟更低

常见“断链”原因

- 发生重分区（如 keyBy）
- 并行度不同
- 显式禁用 chaining
- 某些算子语义/实现不允许链化

图 6-1 Flink 会尽量把可链化的相邻算子合并成一个 Task

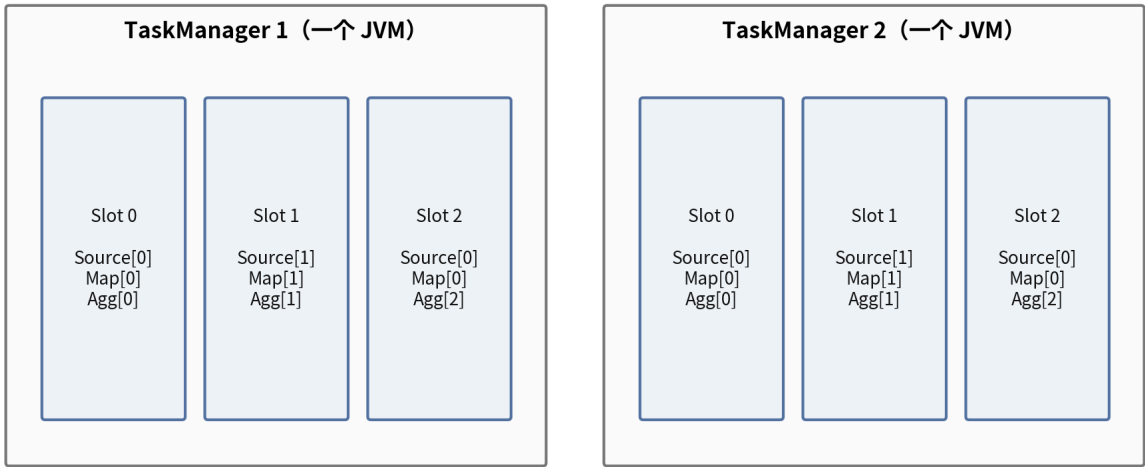
如果 Source、map、filter 等算子之间是 one-to-one 数据交换，并且满足链化条件，Flink 可以把多个 Operator 的对应 Subtask 链接到同一个 Task 中，由同一个线程顺序执行。这叫 Operator Chaining。

- 好处 1：减少线程之间的数据交接。
- 好处 2：减少缓冲和序列化开销。
- 好处 3：通常可以提高吞吐并降低延迟。
- 代价：链在一起后，Web UI 中的运行单元可能不再和你代码里的每一个 map/filter 一一对应；排查时要学会看 Operator Chain。

一句话区分 Operator 是逻辑计算节点；Subtask 是它的并行实例；多个相邻 Subtask 还可能被 chain 成一个运行时 Task。

7. TaskManager 与 Task Slot：真正执行任务的 Worker

TaskManager、Task Slot 与 Slot Sharing



默认 Slot Sharing 下，一个 Slot 可以承载同一 Job 中多个不同算子的 Subtask；Slot 不是“一个算子 = 一个槽”。

图 7-1 TaskManager 是 JVM Worker，Task Slot 是资源调度单位

TaskManager 是真正执行 Flink 数据流 Task 的 Worker 进程。一个 TaskManager 通常是一个 JVM 进程，内部可以运行多个 Task/Subtask。为了控制它接收多少工作，Flink 使用 Task Slot 作为资源调度单位。

对象	本质	关键点
TaskManager	Worker / JVM 进程	执行 Task，缓存并交换数据流
Task Slot	TaskManager 内部的资源份额/调度单位	不是一台机器，也不是一个 CPU Core
Slot Sharing	同一 Job 的不同 Task/Subtask 可共享一个 Slot	一个 Slot 可以承载一整条 pipeline 的若干 Subtask
Thread	Task 实际执行线程	Operator Chaining 后多个算子可在同一 Task 线程中执行

容易考错的地方 “3 个 Slot = 只能运行 3 个算子” 是错的。默认 Slot Sharing 下，同一 Job 中不同算子的 Subtask 可以共享 Slot；官方文档甚至指出，集群所需 Slot 数往往接近 Job 使用的最高并行度，而不是所有算子 Subtask 数量之和。

8. JobManager 不是单一职责：Dispatcher、JobMaster、ResourceManager

为了便于入门，我们常把“协调者”统称为 JobManager。但在当前 Flink 架构中，JobManager 进程内部包含几个重要职责角色。

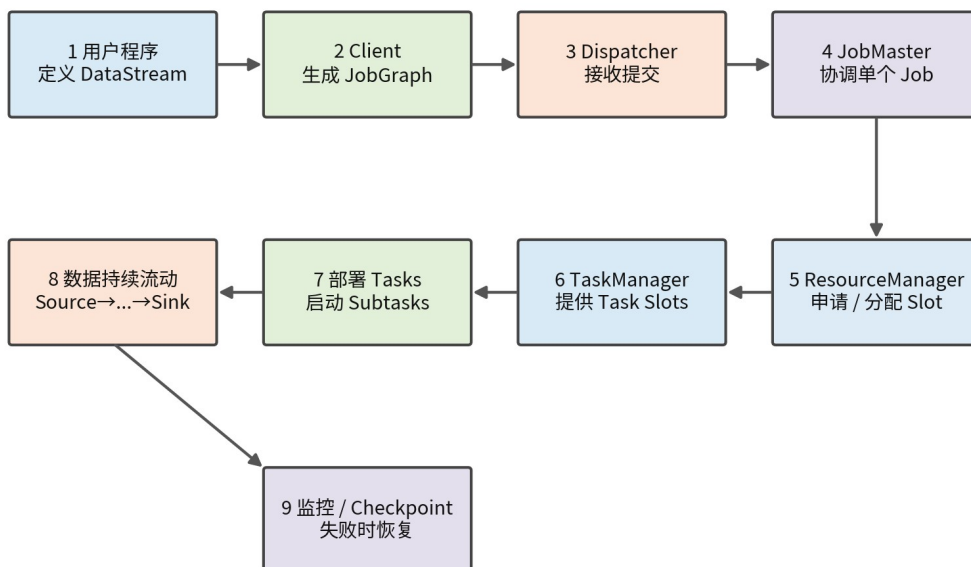
组件	主要职责	通俗比喻
----	------	------

Dispatcher	提供 REST 提交入口，为提交的 Job 启动 JobMaster，并承载 Web UI	接待/受理中心
JobMaster	负责单个 Job 的执行管理：调度、状态、失败反应等	某个项目的项目经理
ResourceManager	负责 Slot/资源的申请、分配与回收；与 YARN/K8s/Standalone 资源环境对接	资源管理员
TaskManager	执行实际 Task	工人/生产车间

与 Hadoop YARN 对比时要谨慎 Flink 自己也有一个叫 ResourceManager 的组件，它和 YARN ResourceManager 不是同一个对象。Flink 部署在 YARN 上时，Flink 的资源管理逻辑还会与外部 YARN 资源框架协作。

9. 一个 Flink Job 提交以后，到底发生了什么

一个 Flink Job 从“代码”到“分布式执行”的控制路径



简化模型：实际内部还有 ExecutionGraph、调度器、部署描述等结构，但初学阶段先抓住角色与数据流。

图 9-1 Flink Job 从用户代码到分布式执行的简化生命周期

- ① 你写用户程序：定义 Source、Transformation、keyBy、Window/State、Sink。
- ② Client 执行程序的构图部分，把数据流描述整理成 JobGraph（可理解为可提交的作业图）。
- ③ Client 把 Job 提交给 Flink 集群入口。
- ④ Dispatcher 接受提交，并为这个 Job 启动/关联一个 JobMaster。
- ⑤ JobMaster 根据作业拓扑与并行度，确定需要部署哪些并行执行实例。
- ⑥ ResourceManager 协调可用 Task Slots；资源不足时，在 YARN/Kubernetes 等部署方式下可向外部资源提供者申请资源。

- ⑦ TaskManager 提供 Slots，JobMaster 将具体 Tasks 部署到这些 TaskManager 上。
- ⑧ Source、Operator、Sink 对应的 Task/Subtask 开始长期运行，数据持续流动。
- ⑨ JobMaster 持续监控运行状态，并参与 Checkpoint 协调、失败处理和恢复。

10. 贯穿案例：实时统计每个用户累计支付金额

```
// 概念代码（经典 DataStream 思路）
DataStream<Order> orders = source(...);

orders
    .filter(o -> o.status.equals("paid"))
    .keyBy(o -> o.userId)
    .map(...)           // 或 reduce/process 等状态计算
    .sinkTo(...);

env.execute("user-spend");
```

假设并行度设置为 3。事件会按 userId 经过 keyBy 路由：用户 A 的所有订单始终交给同一个 Keyed Subtask，用户 B 也会稳定进入自己的目标 Subtask。这样那个 Subtask 才能维护“这个用户当前累计消费了多少”的 Keyed State。

事件	userId	金额	keyBy 后的逻辑结果
e1	A	100	发送到负责 A 的 Keyed Subtask
e2	B	50	发送到负责 B 的 Keyed Subtask
e3	A	30	仍发送到和 e1 相同的 Subtask
e4	C	80	根据 Key 路由到某个目标 Subtask

为什么 keyBy 对 State 至关重要 如果同一个用户的事件随机跑到不同 TaskManager，那么每个地方只看到一部分历史，状态就会被拆散。keyBy 让“同一个 Key 的历史”可以集中由一个逻辑并行实例管理。

11. 并行度与 Slot：到底需要多少资源

初学者经常把“所有算子的 Subtask 数相加”当成需要的 Slot 数。默认 Slot Sharing 下，这通常会严重高估。

经验心智模型 在默认 Slot Sharing 下，一个 Job 通常至少需要能够覆盖其最高并行度的 Slot 数。例如主要算子最高并行度是 4，那么 4 个 Slot 往往就能承载同一 Job 的一整套 pipeline，而不是 Source 4 + Map 4 + Window 4 = 12 个 Slot。

2 个 TaskManager × 每个 2 Slots	集群总计 4 个 Slots
Job 主要算子 Parallelism = 4	理论上可让 4 个并行流水线同时占用 4 个 Slot
某算子 Parallelism = 8	若可用 Slot 仍只有 4，资源/调度策略不足时无法像“8 个完整独立槽位”那样同时展开；具体行为取决于资源与部署配置

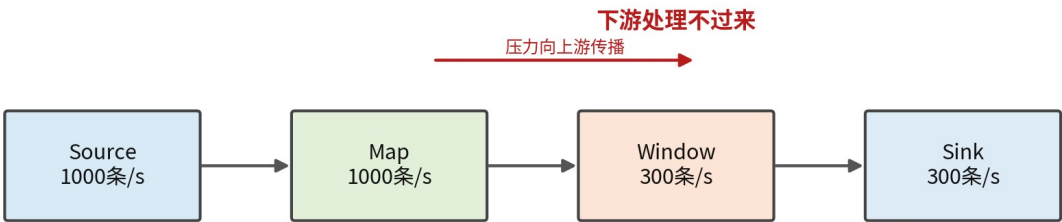
注意：Slot 的资源语义不等价于 CPU 隔离。官方文档明确说明，Task Slot 主要用于 TaskManager 资源切分/托管内存隔离；不能简单说“1 Slot = 1 CPU Core”。

12. 三不要混：据流、JobGraph、运行时 Task 图

你看到的层次	回答的问题	例子
业务 / API 数据流	“我要做什么？”	Source → filter → keyBy → window → Sink
作业图 JobGraph（提交层）	“这个 Job 的可调度结构是什么？”	哪些节点、边、并行度、链化关系
运行时 Task/Subtask	“现在到底在哪里执行？”	TaskManager 1 的 Slot 0 正在跑某个 chain 的 Subtask 2

科研/排错视角 业务逻辑错时看第一层；性能和 Shuffle 问题看数据交换与并行度；资源、失败、反压问题则需要继续下钻到 TaskManager、Subtask、Slot、日志和 Web UI。

13. 一个重要预告：为什么流式系统会出现 Backpressure



Backpressure 不是“程序出错”，而是流式系统在告诉你：下游消费速度低于上游生产速度。

图 13-1 下游处理速度低于上游生产速度时，压力会逐渐向上传播

在 MapReduce/Spark 批处理中，慢任务通常表现为“某个 Stage 很久不结束”。在 Flink 的长期流任务中，如果下游长期处理不过来，上游不能无限制地把数据堆在内存里，于是会出现 Backpressure（反压）。

- 它首先是一个流量/吞吐不匹配信号，不等于程序异常。
- 可能原因包括 Sink 太慢、某个 key 热点、外部数据库响应慢、并行度不足、状态/窗口计算太重。
- 后续课程会通过 Flink Web UI、Busy/Backpressured/Idle 等指标具体分析。

14. 以后看 Flink Web UI 时，你应该找什么

Web UI 对象	你应该问的问题
Job / JobGraph	数据流拓扑是什么？哪些 Operator 被链在一起？
Parallelism	每个节点有多少 Subtask？是否符合预期？
Subtasks	是不是某一个 Subtask 特别慢？
TaskManagers	任务到底分布在哪些 Worker 上？
Task Slots	资源是否足够？Slot 是否被占满？
Metrics / Backpressure	下游是不是处理不过来？是否存在热点或 I/O 瓶颈？

观察原则 不要只看 Job 是 RUNNING 还是 FAILED。Flink 的价值恰恰在于 Job 可能运行几天、几个月；真正重要的是持续观察吞吐、延迟、状态大小、Checkpoint 和反压。

15. 与你当前 Hadoop 虚拟机有关的版本提醒

你当前 Hadoop 教学环境使用 JDK 1.8。这里需要提前知道一个现实变化：当前 Flink 2.x 稳定文档已经不再支持 Java 8；官方推荐 Java 17，Java 11 也仍在兼容列表中。因此后续真正安装当前 Flink 时，不建议直接替换 Hadoop 原来的 JDK 1.8。

后续上机策略 更稳妥的做法是：在同一 Ubuntu VM 中并存 JDK 1.8 与 JDK 17，Hadoop 继续使用原来的 JDK 1.8；Flink 启动脚本单独指定 Java 17。这样不会为了 Flink 把前面已经稳定的 Hadoop 环境破坏掉。

本讲先学习跨版本稳定的执行模型，不要求你现在改环境。真正安装 Flink 时我们再单独做一节“版本与 Java 共存”的保姆式教程。

16. 初学者最容易混淆的 10 个问题

1. DataStream 是不是把所有数据放在内存里？

不是。它主要是对持续数据流及其计算关系的抽象；数据被分区并在不同 Subtask 之间持续传输。

2. 一个 Operator 就是一个线程吗？

不是。Operator 可以有多个 Subtask；多个相邻 Operator 的 Subtask 又可能被 chain 到同一个 Task 线程。

3. Parallelism=4 是不是需要 4 台机器？

不是。4 个 Subtask 可以分布在 1 台或多台 TaskManager 上，取决于 Slot 和部署资源。

4. 一个 Slot 只能运行一个 Operator 吗？

不是。默认 Slot Sharing 下，同一 Job 的不同 Task/Subtask 可以共享 Slot。

5. 一个 Slot 就等于一个 CPU Core 吗？

不能这么等同。Slot 是 Flink 的资源调度单位，官方特别说明它不提供简单的 CPU 隔离语义。

6. keyBy 会不会把数据“收集成一个 List”？

不会。它持续按 Key 路由事件，让相同 Key 进入同一个下游逻辑 Subtask。

7. keyBy 一定跨网络吗？

语义上属于 redistributing。物理上记录是否真的跨机器取决于上下游 Subtask 的实际部署位置。

8. JobManager 是不是自己处理业务数据？

主要不是。它协调执行；真正执行算子和交换数据的是 TaskManagers。

9. Flink Job 什么时候算完？

有界流会结束；典型无界流 Job 会长期运行，除非被取消、失败或输入终止。

10. Operator Chaining 会不会改变业务结果？

正常情况下不改变语义，它主要改变运行时组织方式与性能。

17. 练习：自己推演一个实时订单 Job

业务需求：Kafka 持续产生订单事件，需要过滤掉未支付订单，按 shopId 统计每个店铺累计销售额，并把结果写入下游数据库。假设 Source P=2，聚合 P=4，Sink P=2。

- 问题 A：请画出 Source → filter → keyBy(shopId) → aggregate → Sink。
- 问题 B：哪些边更可能是 one-to-one？哪一处一定具有“重分区”语义？
- 问题 C：aggregate P=4 表示什么？
- 问题 D：为什么同一个 shopId 必须稳定进入同一个 Keyed Subtask？
- 问题 E：如果 filter 与 Source 可链化，运行时 Task 数为什么可能少于代码中的 Operator 数？
- 问题 F：如果 Sink 数据库只能处理 200 条/s，而上游长期输出 2000 条/s，会出现什么系统现象？

答案提示 B: keyBy 是 redistributing；C: 聚合算子有 4 个并行 Subtask；D: 否则状态会被拆散；E: Operator Chaining；F: 下游成为瓶颈并逐渐形成 Backpressure。

18. 本讲最后压缩成 8 句话

- ① Flink DataStream 程序是一张长期运行的数据流图。
- ② Operator 是逻辑计算步骤，Parallelism 决定它有多少并行 Subtask。
- ③ Stream Partition 是并行数据流的分片；数据可以 one-to-one 传递，也可以重新分区。
- ④ keyBy 的核心作用是按 Key 持续路由，为 Keyed State 和按 Key 计算建立基础。
- ⑤ 多个可链化 Operator 的 Subtask 可以合并为一个 Task，由一个线程执行。

- ⑥ TaskManager 是真正执行 Task 的 Worker；Task Slot 是资源调度单位，不等于一个算子或一个 CPU Core。
- ⑦ JobManager 侧负责提交、资源协调、Job 调度和故障/Checkpoint 协调，TaskManager 侧负责真正跑数据流。
- ⑧ 以后分析 Flink 性能，要把“业务数据流、并行 Subtask、Slot 资源、网络重分区”四层一起看。

下一讲 Flink 第 3 讲将进入流处理最核心、也最反直觉的主题：Event Time、Processing Time、乱序事件与 Watermark。我们会用“迟到的订单事件”一步一步解释为什么不能只看机器当前时间。

参考：本讲采用的官方概念口径

- Apache Flink - Flink Architecture：JobManager、TaskManager、Task Slot、Operator Chaining、Slot Sharing。
- Apache Flink - Parallel Execution：Parallelism 与并行 Subtask 的定义。
- Apache Flink - Learn Flink Overview：one-to-one 与 redistributing streams。
- Apache Flink - Deployment Overview：Client 生成 JobGraph 并提交给 JobManager；TaskManager 执行 operators。
- Apache Flink - Java Compatibility：Flink 2.x 默认/推荐 Java 17，Java 8 已不在当前支持列表。

说明：DataStream API V2 在当前官方文档中仍标为 experimental。本课程本阶段先使用跨版本稳定的经典 DataStream 执行概念，避免把学习重点放在尚未稳定的 API 变化上。